Лабораторная работа №7
Тема. Методы оптимизации в нейросетевых вычислениях
(8-11 баллов)

ПРИВЯЗКА К ТЕОРИИ, ЗАДАНИЕ, КРИТЕРИИ ОЦЕНИВАНИЯ, ПРИМЕР РЕШЕНИЯ (2 варианта)

Вопросы зачёта:
23. Бенчмарк-функции («искусственные ландшафты»): функции Химмельблау, Розенброка. Эффективность методов оптимизации для таких функций.
24. Проблема седловых точек при решении задачи оптимизации функции нескольких переменных.
25. Момент Нестерова. Эвристики для градиентного спуска, применяемые в нейросетевых вычислениях.

Задание 1 (3 балла). Исследуйте функцию нескольких переменных по варианту в заданной области аналитически: определите седловые точки, локальные экстремумы, глобальный минимум. Напишите программный модуль для построения линий уровня этой функции. Сделайте выводы, какие сложности для оптимизационного алгоритма могут быть в случае такой функции.
Оценивание:
- указаны все седловые точки, исследование на локальные экстремумы (указаны все / доказано, что нет) – 1 балл,
- найден глобальный минимум – 1 балл,
- построены линии уровня (программно), которые иллюстрируют выводы выше – 1 балл.

Задание 2 (5 баллов, по 1 баллу за каждый пункт, в п.3 – 2 балла).
1. Выберите начальное приближение, скорость обучения, критерий останова так, чтобы последовательные приближения градиентного спуска сходились к седловой точке за ~100 итераций (помимо критериев из предыдущих разделов – норма градиента близка к нулю с заданной точностью, значения функции близки с заданной точностью – здесь важно зафиксировать количество итераций, так как оно будет использоваться далее). Сделайте визуализацию линий уровня и последовательных приближений. Начальное приближение, седловую точку, количество итераций из этого пункта используйте во всех последующих.
2. Реализуйте программно эвристики для градиентного спуска (метод по варианту). С помощью реализованных эвристик получите за то же количество итераций последовательность приближений, которая «преодолевает» седловую точку.
3. Подберите область отображения для эффективной иллюстрации диаграммы последовательных приближений. Не меняя область отображения, найдите значения гиперпараметров:
(1) в области явно просматривается движение в стороны от направления к глобальному минимуму (пилообразная ломаная),
(2) в области направление более чёткое без движений в стороны.
4. Используйте готовую реализацию одной из эвристик для градиентного спуска (рекомендуются функции PyTorch: https://docs.pytorch.org/docs/stable/optim.html). Убедитесь, что алгоритм успешно решает задачу «преодоления» седловой точки и постройте один вариант визуализации диаграммы последовательных приближений (в области из п.3)

Пример решения. Задания 1 и 2. Вариант 0.

Дано по варианту:

$$f(x,y)=10^{-2}\left(10x^2-2xy-61x+14y-63\right),\ (x,y)\in[-20,20]\times[-50,50]$$

Задание 1:

Область определения функции без ограничения: ООФ $f(x,y)=\mathrm{R}^2$. Частные производные первого порядка существуют в каждой точке области определения. Область определения функции с учетом ограничения: $[-20,20]\times[-50,50]$.

Стационарные точки: $\left\{\left(7,\ 39.5\right)\right\}$, так как:

$$\begin{cases} f'_x=0 \\ f'_y=0 \end{cases} \Rightarrow \begin{cases} f'_x=10^{-2}\left(20x-2y-61\right)=0 \\ f'_y=10^{-2}\left(-2x+14\right)=0 \end{cases} \Leftrightarrow \begin{cases} y=10x-30.5 \\ x=7 \end{cases} \Leftrightarrow \begin{cases} y=39.5 \\ x=7 \end{cases}.$$

Определение типа стационарной точки (частные производные второго порядка, в том числе смешанные, существуют во всех точках области определения функции) $\left(7,\ 39.5\right)$:

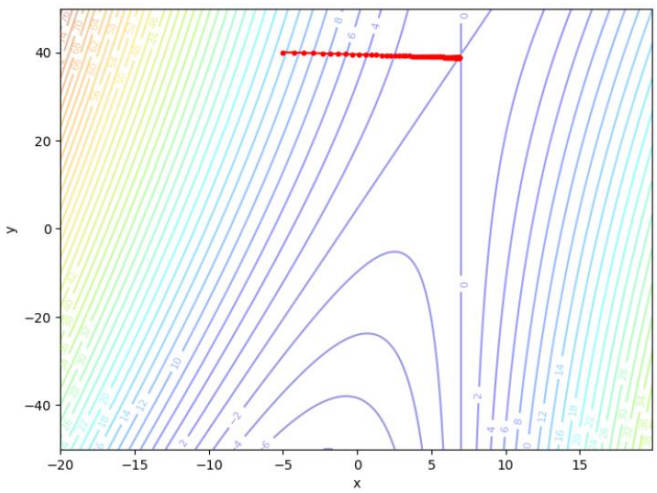
$$H=\begin{pmatrix} \dfrac{\partial^2 f}{\partial x^2} & \dfrac{\partial^2 f}{\partial y\partial x} \\ \dfrac{\partial^2 f}{\partial x\partial y} & \dfrac{\partial^2 f}{\partial y^2} \end{pmatrix}=10^{-2}\cdot\begin{pmatrix} 20 & -2 \\ -2 & 0 \end{pmatrix}\Rightarrow D_1=0.2, D_2=-0.0004$$ -- гессиан

неопределенный, значит имеем седловую точку. Седловая точка (x,y): (7.0, 39.5).
Локальных экстремумов нет. Глобальный минимум будем искать на границе области определения функции: … [необходимо выполнить!]

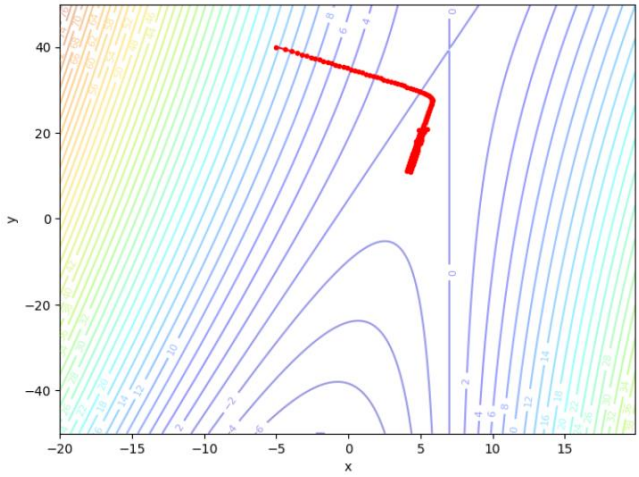
Задание 2:
1. Выберите начальное приближение, скорость обучения, критерий останова так, чтобы последовательные приближения градиентного спуска сходились к седловой точке за ~100 итераций. Сделайте визуализацию линий уровня и последовательных приближений.

Начальное приближение: (x_0,y_0) = (-5.0, 40.0), пороги для критерия останова зададим сами.
Скорость обучения и критерий останова: $\alpha=0.3,\ \left|f(x^{it+1},y^{it+1})-f(x^{it},y^{it})\right|<\varepsilon=10^{-5}$.

№ итерации, на которой выполнился критерий останова: 98.
Код для визуализации – см. доп материал к ЛР4 (Contours and solutions.ipynb)

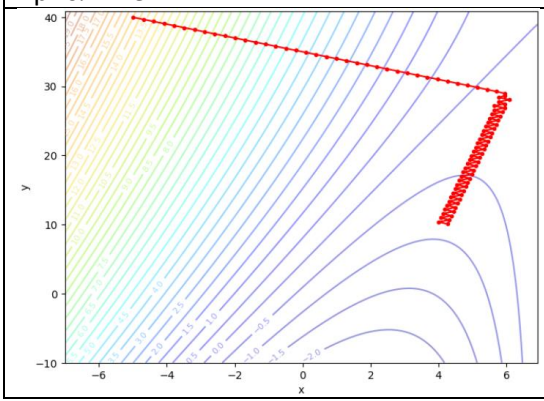
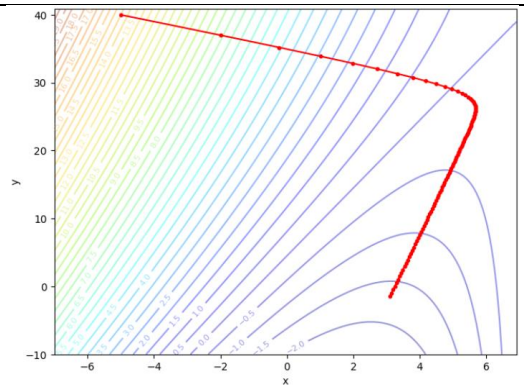


2. Реализуйте программно эвристики для градиентного спуска (RMSProp). С помощью реализованных эвристик получите за то же количество итераций последовательность приближений, которая «преодолевает» седловую точку, стартуя с начального приближения из п.1. Метод RMSProp, скорость обучения α и параметр модификации скорости γ: $\alpha = 0.3, \gamma = 0.75$



3. Подберите область отображения для эффективной иллюстрации диаграммы последовательных приближений. Не меняя область отображения, найдите значения гиперпараметров, при которых за 100 итераций:
(1) в области явно просматривается движение в стороны от направления к глобальному минимуму (пилообразная ломаная),
(2) в области направление более чёткое без движений в стороны.

$(x, y) \in [-7.0,\ 7.0] \times [-10.0,\ 41.0]$ -- область локализации ломаной последовательных приближений (98 итераций).

Метод RMSProp, $\alpha = 0.3, \gamma = 0.1$ Пилообразная ломаная последовательных приближений	Метод RMSProp, $\alpha = 0.3, \gamma = 0.99$ Движение без пилообразных отклонений
	

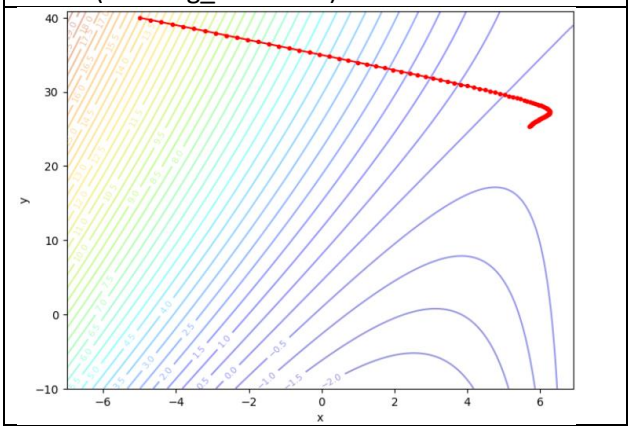
4. Используйте готовую реализацию одной из эвристик для градиентного спуска (рекомендуются функции PyTorch). Убедитесь, что алгоритм успешно решает задачу «преодоления» седловой точки и постройте один вариант визуализации диаграммы последовательных приближений (в области из п.3, кол-ве итераций из п.1)

```python
import torch

n_iter = 100
learning_rate = 0.3
step_x = 0.1
step_y = 0.1
bounds = np.asarray([[-7.0, 7.0], [-10.0, 41.0]])
history = {}

solution = torch.tensor([[-5.0], [40.0]], requires_grad=True)
optimizer = torch.optim.Adam([solution], lr=learning_rate)
trace = []

for i in range(n_iter):
    optimizer.zero_grad()
    score = objective(solution[0], solution[1])
    score.backward()
    trace.append(solution.clone().detach().numpy())
    optimizer.step()
for it in range(len(trace)):
  history[it] = {'solution': (trace[it][0,0], trace[it][1,0]), 'score': None}

plot_history('Adam_history', objective, bounds, step_x, step_y, history)
```

Adam (learning_rate = 0.3)